

Chapter 11

Retrievers

The fourth core RAG component — how LangChain fetches the most relevant documents for a query, and the advanced strategies (MMR, Multi Query, Contextual Compression) that make retrieval smarter.

WHAT THIS CHAPTER COVERS

- ▶ What a Retriever is and why it matters for RAG
- ▶ Retrievers as Runnables
- ▶ Categorizing retrievers by data source vs. search strategy
- ▶ Wikipedia Retriever
- ▶ Vector Store Retriever
- ▶ Maximum Marginal Relevance (MMR)
- ▶ Multi Query Retriever
- ▶ Contextual Compression Retriever

Retrievers

RECAP — CHAPTER 10

Chapter 10 covered Vector Stores — how documents are converted into embeddings and stored for similarity search, using Chroma DB as the running example with its Tenant → Database → Collection → Document hierarchy. With storage and indexing in place, the next question is: how does an application actually pull the right documents out of that store in response to a user query? That is the job of the Retriever, the fourth core RAG component covered in this chapter.

1. What Are Retrievers?

Definition

A *Retriever* is a component in LangChain that fetches relevant documents from a data source in response to a user's query. It sits between the user's question and the raw data source, deciding which pieces of stored information are worth returning.

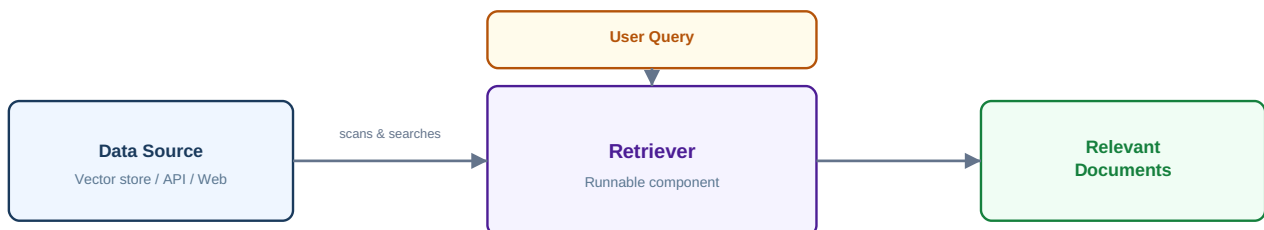


Figure 11.1 — A retriever accepts a user query, searches a data source internally, and returns the most relevant documents.

Why It Exists

Any RAG-based application needs a mechanism to go from "user typed a question" to "here are the documents that answer it." Without a standardized abstraction, every data source (a vector store, an API, a website) would need its own custom search logic wired directly into the application. Retrievers give LangChain a single, consistent interface for this job regardless of what is being searched.

How It Works

1. A data source holds all the available data — this could be a vector store, an external API, or a scannable website.
2. The user sends a query to the system.
3. The retriever receives this query and internally scans the data source to determine which documents are most relevant.
4. The retriever fetches those relevant documents and returns them.

In its simplest form, a retriever behaves like a function: it takes a user query as input and returns multiple **Document** objects as output. Internally it behaves like a search engine, deciding relevance and returning ranked results.

KEY PROPERTIES OF RETRIEVERS

1. **Not a single retriever** — LangChain provides many different retrievers (roughly 20–30), each suited to different use cases.
2. **All retrievers are Runnable** — just like Models and Prompts, every retriever exposes an `.invoke()` method. This means retrievers can be composed into chains, or plugged directly into existing chains, which greatly increases system flexibility.

IMPORTANT

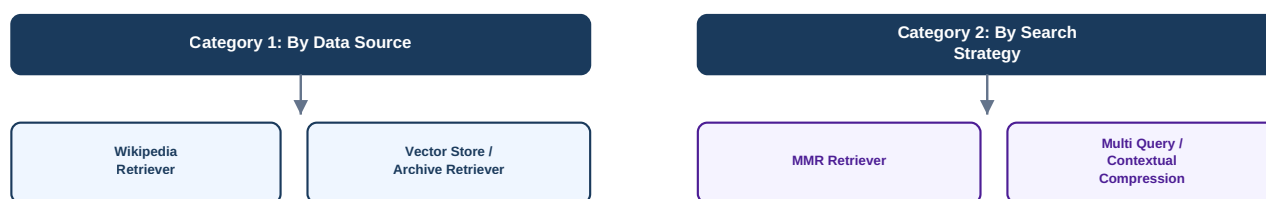
A retriever is not a document loader. Even retrievers that pull from external sources (like Wikipedia) perform some form of intelligent search/ranking internally — they do not simply load everything. That internal decision-making is what qualifies them as retrievers rather than loaders.

🔄 Quick Revision — What Are Retriever?

- A retriever fetches relevant documents from a data source based on a user query.
- Conceptually: input = query, output = list of Document objects.
- LangChain has many retrievers, not just one, each fit for a different scenario.
- Every retriever is a Runnable, exposing `.invoke()` and composable into chains.
- A retriever differs from a document loader because it performs relevance-based search internally, not blind loading.

2. Categorizing Retriever

Since LangChain offers dozens of retrievers, it helps to organize them along two independent axes.



Data-source retrievers differ in WHERE they search; strategy retrievers differ in HOW they search.

Figure 11.2 — Retriever can be categorized by the data source they search or by the search strategy they use.

1. BY DATA SOURCE

Different retrievers are built to work with different underlying data sources:

- **Wikipedia Retriever** — searches Wikipedia articles.

- **Vector Store Retriever** — searches documents stored in a vector store.
- **Arxiv Retriever** — scans research papers on the Arxiv website.

2. BY SEARCH STRATEGY

Other retrievers are differentiated by the internal mechanism they use to decide relevance, regardless of the data source:

- **MMR (Maximum Marginal Relevance)** — balances relevance with diversity.
- **Multi Query Retriever** — resolves ambiguity by generating multiple query variations.
- **Contextual Compression Retriever** — trims retrieved documents to only the relevant portions.

This chapter covers exactly these five retrievers in detail. Dozens of others exist (Parent Document Retriever, Time-Weighted Vector Retriever, Self-Query Retriever, Ensemble Retriever, Multi Retriever, and more) and can be looked up in the official LangChain documentation as needed for a specific project.

Quick Revision — Categorizing Retriever

- Retriever can be grouped by the data source they operate on (Wikipedia, vector store, Arxiv, etc.).
- Retriever can also be grouped by their search strategy (MMR, Multi Query, Contextual Compression, etc.).
- These two axes are independent; a retriever's category depends on what question you are asking about it.
- LangChain documentation lists all available retrievers with sample code for each.

3. Wikipedia Retriever

Definition

The Wikipedia Retriever is a retriever that queries the Wikipedia API to fetch relevant content for a given query.

How It Works

1. The user provides a query, e.g. a topic they want Wikipedia articles about.
2. The retriever sends this query to the Wikipedia API.
3. Internally, it uses **keyword matching** (not semantic search) to decide which articles are most relevant — the more keywords that match, the higher the relevance.
4. The most relevant articles are fetched and returned as LangChain **Document** objects.

IMPORTANT

The Wikipedia Retriever performs keyword-based matching internally, not semantic/embedding-based search. This is a key distinguishing detail from vector-store-based retrievers.

Example

```
from langchain_community.retrievers import WikipediaRetriever

# top_k_results: how many articles to return; lang: result language
retriever = WikipediaRetriever(
    top_k_results = 2,
    lang = "en"
)

query = "The geopolitical history of India and Pakistan from the perspective of Chinese."

# .invoke() works because WikipediaRetriever is a Runnable
docs = retriever.invoke(query)

for i, doc in enumerate(docs):
    print(f"\n --- {i+1} ---")
    print(f"Content:\n{doc.page_content}...")
```

The query intentionally contains several distinct keywords (India, Pakistan, geopolitical history) so the keyword-matching mechanism has plenty to work with. Calling `retriever.invoke(query)` sends the request to the Wikipedia API behind the scenes, performs keyword matching across articles, and returns the top-ranked ones as Document objects with page content and metadata.

Quick Revision — Wikipedia Retriever

- Queries the Wikipedia API for relevant articles given a user query.
- Uses keyword matching internally, not semantic search.
- `top_k_results` controls how many articles are returned; `lang` sets the article language.
- Returns results as standard LangChain Document objects.
- It is a retriever (not a loader) because it performs relevance ranking, not blind loading.

4. Vector Store Retriever

Definition

A Vector Store Retriever is the most common type of retriever in LangChain. It lets you search and fetch documents from a vector store based on semantic similarity, using vector embeddings.

How It Works

1. Documents are stored in a vector store (Chroma, FAISS, Weaviate, etc.).
2. Each document is converted into a dense vector using an embedding model.

3. The user's query is also converted into a vector using the same embedding model.
4. The query vector is compared against all stored document vectors using semantic similarity.
5. The top-K most similar documents are fetched and returned.

Example

```
from langchain_community.vectorstores import Chroma
from langchain_nvidia_ai_endpoints import NVIDIAEmbeddings
from langchain_core.documents import Document

embedding = NVIDIAEmbeddings(model = "nvidia/nemotron-3-embed-1b")

documents = [
    Document(page_content="LangChain helps developers build LLM applications easily."),
    Document(page_content="Chroma is a vector database optimized for LLM-based search."),
    Document(page_content="Embeddings convert text into high-dimensional vectors."),
    Document(page_content="OpenAI provides powerful embedding models."),
]

# Vector store is created and documents are embedded + added in one step
vector_store = Chroma.from_documents(
    documents = documents,
    embedding = embedding,
    collection_name = "my_collection"
)

# .as_retriever() turns the vector store into a Runnable retriever
retriever = vector_store.as_retriever(search_kwargs = {"k": 2})

query = "What is Chroma used for?"
results = retriever.invoke(query)

for i, doc in enumerate(results):
    print(f"\n--{i+1}--")
    print(f"\n--{doc}--")
```

`vector_store.as_retriever()` wraps the vector store in a retriever interface. The `k` parameter controls how many top results are returned.

WHY NOT JUST USE THE VECTOR STORE DIRECTLY?

A natural question is: since a vector store already exposes `similarity_search()`, why wrap it in a retriever at all? Calling `vector_store.similarity_search(query, k=2)` directly produces the same output as the retriever above.

IMPORTANT

A vector store can only perform similarity search using a single fixed strategy — comparing vectors by a distance metric and returning the top matches. A retriever built from that vector store, however, can be swapped in for far more advanced search strategies (MMR, Multi Query, Contextual Compression) without changing how the rest of the application calls it. The retriever is also a Runnable, so it plugs directly into chains — the vector store's raw method does not.

🔄 Quick Revision — Vector Store Retriever

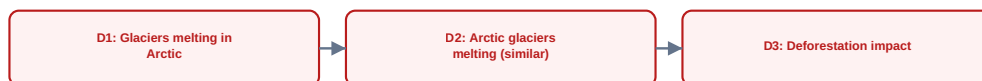
- Most common retriever type; performs semantic similarity search over a vector store.
- Documents and queries are both embedded into the same vector space for comparison.
- Created via `vector_store.as_retriever(search_kwargs={'k': N})`.
- Functionally similar to calling `vector_store.similarity_search()` directly for basic cases.
- Its real value is that it is a Runnable and can be swapped for advanced search strategies (MMR, Multi Query, etc.) without changing calling code.

5. Maximum Marginal Relevance (MMR)

The Problem: Redundant Results

A plain similarity search retriever can return documents that are all highly relevant but say almost the same thing. For example, if a data source has five documents about climate change and two of them both describe glaciers melting in near-identical language, a query like "What are the adverse effects of climate change?" might return both of those near-duplicate documents among the top results — wasting a result slot that could have surfaced a different perspective (deforestation, wildfires, coastal flooding, etc.).

NORMAL SIMILARITY SEARCH (redundant)



MMR SEARCH (relevant + diverse)



MMR picks the most relevant document first, then favors documents that are relevant but dissimilar to what is already picked.

Figure 11.3 — Normal similarity search can return redundant, near-duplicate documents, while MMR favors both relevance and diversity.

Definition

Maximum Marginal Relevance (MMR) is an information retrieval algorithm designed to reduce redundancy in retrieved results while maintaining high relevance to the query. Its core philosophy is: pick results that are not only relevant to the query, but also different from each other.

How It Works

1. MMR first picks the single most relevant document to the query.
2. For each subsequent pick, it selects a document that is both relevant to the query **and** maximally dissimilar to the documents already selected.
3. This process repeats until the requested number of results is reached.

THE LAMBDA PARAMETER

MMR search is controlled by a **lambda** (**lambda_mult**) parameter ranging from 0 to 1:

lambda_mult value	Behavior
1	MMR behaves exactly like normal similarity search (no diversity preference).
0	Maximum diversity is prioritized over relevance.
Between 0 and 1	A balance between relevance and diversity — this is the typical operating range.

Example

```
from langchain_core.documents import Document
from langchain_community.vectorstores import FAISS
from langchain_nvidia_ai_endpoints import NVIDIAEmbeddings

embedding_model = NVIDIAEmbeddings(model = "nvidia/nemotron-3-embed-1b")

docs = [
    Document(page_content="LangChain makes it easy to work with LLMs."),
    Document(page_content="LangChain is used to build LLM based applications."),
    Document(page_content="Chroma is used to store and search document embeddings."),
    Document(page_content="Embeddings are vector representations of text."),
    Document(page_content="MMR helps you get diverse results when doing similarity search."),
    Document(page_content="LangChain supports Chroma, FAISS, Pinecone, and more."),
]

vectorstore = FAISS.from_documents(documents = docs, embedding = embedding_model)

# search_type="mmr" enables Maximum Marginal Relevance instead of plain similarity
retriever = vectorstore.as_retriever(
    search_type = "mmr",
    search_kwargs = {"k": 3, "lambda_mult": 0.5} # lambda_mult = relevance-diversity balance
)

query = "What is Langchain?"
result = retriever.invoke(query)

print(result)
```

Setting `search_type="mmr"` switches the retrieval strategy away from plain similarity search. With `lambda_mult=1`, MMR behaves identically to similarity search and may return near-duplicate results (e.g. two very similar "LangChain is used for X" documents). Lowering `lambda_mult` toward 0.5 causes MMR to swap out a redundant result for a more diverse one covering a different aspect of the topic (e.g. embeddings instead of a repeat of the LangChain-definition document).

BEST PRACTICE

Start with `lambda_mult` around 0.5 and adjust based on observed result quality — pushing toward 0 if results still feel redundant, or toward 1 if diverse results feel less relevant.

Quick Revision — MMR

- MMR reduces redundancy in retrieved results while keeping them relevant to the query.
- It first picks the most relevant document, then picks documents that are relevant yet dissimilar to prior picks.
- Enabled via `search_type='mmr'` on a vector store retriever.
- `lambda_mult` (0 to 1) controls the relevance-diversity trade-off; 1 = pure similarity search, 0 = maximum diversity.
- Useful whenever the data source may contain multiple near-duplicate documents on the same topic.

6. Multi Query Retriever

The Problem: Ambiguous Queries

Sometimes a user's query is broad or ambiguous, making it hard to retrieve consistently good documents. For example, "How can I stay healthy?" could mean "What should I eat?", "How often should I exercise?", or "How can I manage stress?" — each interpretation points to different relevant documents. Searching directly with the ambiguous query risks pulling in loosely related or off-topic results.

Definition

The *Multi Query Retriever* resolves ambiguity in a user's query by using an LLM to generate multiple reformulated versions of that query, retrieving documents for each version, and then merging the results.

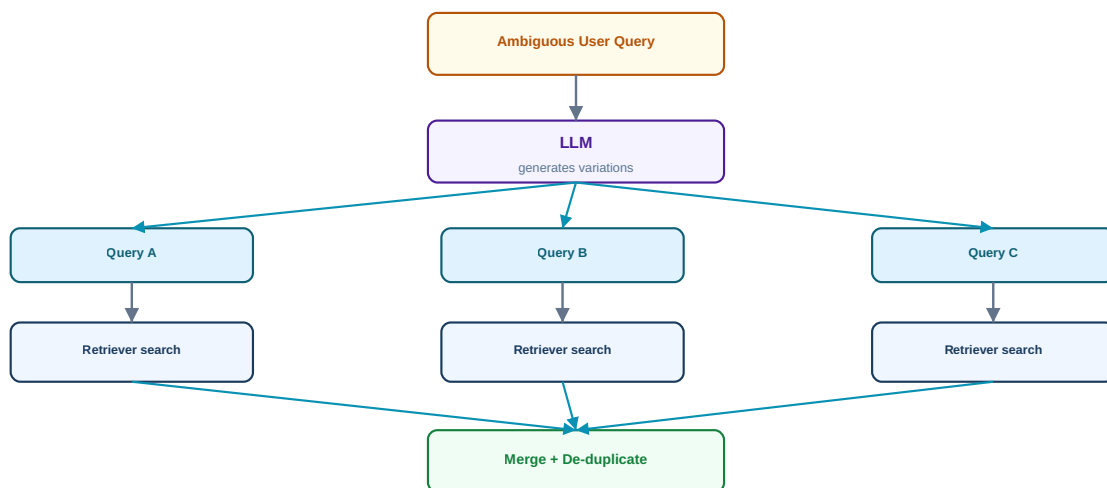


Figure 11.4 — An LLM expands one ambiguous query into several related queries; each is searched independently, and results are merged and de-duplicated.

How It Works

1. The user's (possibly ambiguous) query is sent to an LLM.
2. The LLM generates several diverse but related reformulations of the original query.
3. Each reformulated query is sent independently to a base retriever (e.g. a similarity search retriever).

4. Each search returns its own set of relevant documents.
5. All result sets are merged, duplicates are removed, and the top results are returned to the user.

Example

```

from langchain_classic.retrievers.multi_query import MultiQueryRetriever
from langchain_community.vectorstores import FAISS
from langchain_core.documents import Document
from langchain_nvidia_ai_endpoints import NVIDIAEmbeddings
from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint

embedding_model = NVIDIAEmbeddings(model = "nvidia/nemotron-3-embed-1b")
chat_model = ChatHuggingFace(llm = HuggingFaceEndpoint(repo_id = "Qwen/Qwen2.5-7B-Instruct"))

# Health/lifestyle docs (H1-H5) plus loosely related "distractor" docs (I1-I5)
all_docs = [
    Document(page_content="Regular walking boosts heart health and can reduce symptoms of depression.", metadata={"source": "H1"}),
    Document(page_content="Consuming leafy greens and fruits helps detox the body and improve longevity.", metadata={"source": "H2"}),
    Document(page_content="Deep sleep is crucial for cellular repair and emotional regulation.", metadata={"source": "H3"}),
    Document(page_content="Mindfulness and controlled breathing lower cortisol and improve mental clarity.", metadata={"source": "H4"}),
    Document(page_content="Drinking sufficient water throughout the day helps maintain metabolism and energy.", metadata={"source": "H5"}),
    Document(page_content="The solar energy system in modern homes helps balance electricity demand.", metadata={"source": "I1"}),
    Document(page_content="Python balances readability with power, making it a popular system design language.", metadata={"source": "I2"}),
    Document(page_content="Photosynthesis enables plants to produce energy by converting sunlight.", metadata={"source": "I3"}),
    Document(page_content="The 2022 FIFA World Cup was held in Qatar and drew global energy and excitement.", metadata={"source": "I4"}),
    Document(page_content="Black holes bend spacetime and store immense gravitational energy.", metadata={"source": "I5"})
]

vectorstore = FAISS.from_documents(documents = all_docs, embedding = embedding_model)

# Baseline retriever - plain similarity search
similarity_retriever = vectorstore.as_retriever(search_type = "similarity", search_kwargs = {"k": 5})

# Multi Query Retriever - uses an LLM to expand the query, then delegates to a base retriever
multiquery_retriever = MultiQueryRetriever.from_llm(
    retriever = vectorstore.as_retriever(search_kwargs = {"k": 5}),
    llm = chat_model
)

# Intentionally ambiguous query - "energy" and "balance" overlap with distractor docs
query = "How to improve energy levels and maintain balance?"

similarity_results = similarity_retriever.invoke(query)
multiquery_results = multiquery_retriever.invoke(query)

```

The query deliberately contains words ("energy", "balance") that also appear in unrelated distractor documents about solar systems and Python. The plain similarity retriever gets confused by this overlap and pulls in an off-topic document (e.g. about solar energy systems) among its top-5 results. The Multi Query Retriever, by contrast, expands the query into several health-focused reformulations first, so all five of its final results stay correctly focused on health and nutrition.

REAL WORLD INSIGHT

This is the single biggest benefit of the Multi Query Retriever: it uses the LLM's understanding of intent to correct for surface-level keyword overlap that would otherwise mislead a plain similarity search.

🔄 Quick Revision — Multi Query Retriever

- Solves the problem of ambiguous or broad user queries producing poor retrieval results.
- An LLM generates multiple reformulated versions of the original query.
- Each reformulated query is searched independently using a base retriever.
- All results are merged and de-duplicated before being returned.
- Created via `MultiQueryRetriever.from_llm(retriever=..., llm=...)`.
- More resilient to keyword overlap with unrelated documents than plain similarity search.

7. Contextual Compression Retriever

The Problem: Mixed-Topic Documents

A single stored document can sometimes contain multiple unrelated topics within one chunk — for example, a paragraph about the Grand Canyon that also happens to mention photosynthesis. If a user asks "What is photosynthesis?", a normal retriever correctly identifies this document as relevant and returns it in full — but that means the user also receives irrelevant Grand Canyon content mixed into the answer, degrading the experience.

This kind of mixed-topic document is common in practice: when large source texts are split using a text splitter, chunk boundaries do not always align perfectly with topic boundaries, so a single chunk can end up straddling two unrelated subjects.

Definition

The *Contextual Compression Retriever* is an advanced retriever that improves retrieval quality by compressing documents after retrieval, keeping only the content relevant to the user's query and discarding the rest.

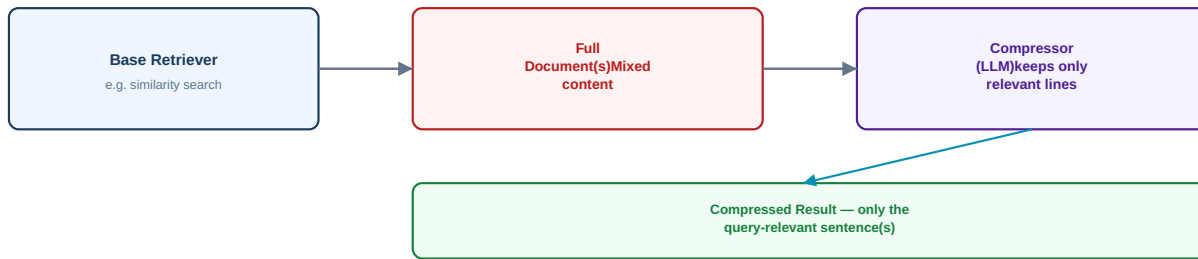


Figure 11.5 — A base retriever fetches full documents; an LLM-based compressor then trims each one down to only the query-relevant content.

How It Works

1. A **base retriever** (e.g. a similarity search retriever) fetches the top-K documents relevant to the query, in full.
2. A **compressor** — usually an LLM — is applied to each retrieved document along with the original query.
3. The compressor keeps only the parts of each document relevant to the query and discards everything irrelevant.
4. The trimmed documents are returned as the final result.

WHEN TO USE IT

- When documents are long and might contain mixed/unrelated information.
- When you want to reduce the context length passed to a downstream LLM.
- When you want to improve the answer accuracy of a RAG pipeline by removing noise.

Example


```

from langchain_community.vectorstores import FAISS
from langchain_nvidia_ai_endpoints import NVIDIAEmbeddings
from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint
from langchain_classic.retrievers import ContextualCompressionRetriever
from langchain_classic.retrievers.document_compressors import LLMChainExtractor
from langchain_core.documents import Document

# Each doc below mixes an unrelated topic with a photosynthesis-related sentence
docs = [
    Document(page_content=(
        """The Grand Canyon is one of the most visited natural wonders in the world.
        Photosynthesis is the process by which green plants convert sunlight into energy.
        Millions of tourists travel to see it every year. The rocks date back millions of
years."""
    ), metadata={"source": "Doc1"}),

    Document(page_content=(
        """In medieval Europe, castles were built primarily for defense.
        The chlorophyll in plant cells captures sunlight during photosynthesis.
        Knights wore armor made of metal. Siege weapons were often used to breach castle
walls."""
    ), metadata={"source": "Doc2"}),

    Document(page_content=(
        """Basketball was invented by Dr. James Naismith in the late 19th century.
        It was originally played with a soccer ball and peach baskets. NBA is now a global
league."""
    ), metadata={"source": "Doc3"}),

    Document(page_content=(
        """The history of cinema began in the late 1800s. Silent films were the earliest
form.
        Thomas Edison was among the pioneers. Photosynthesis does not occur in animal cells.
        Modern filmmaking involves complex CGI and sound design."""
    ), metadata={"source": "Doc4"})
]

embedding_model = NVIDIAEmbeddings(model = "nvidia/nemotron-3-embed-1b")
vectorstore = FAISS.from_documents(docs, embedding_model)

# Base retriever - fetches full documents by similarity
retriever = vectorstore.as_retriever(search_kwargs = {"k": 5})

# Compressor - an LLM that extracts only the query-relevant sentences
llm = ChatHuggingFace(llm = HuggingFaceEndpoint(repo_id = "Qwen/Qwen2.5-7B-Instruct"))
compressor = LLMChainExtractor.from_llm(llm)

# Combine base retriever + compressor into one retriever
compression_retriever = ContextualCompressionRetriever(
    base_retriever = retriever,
    base_compressor = compressor
)

```

```

query = "What is photosynthesis?"
compressed_results = compression_retriever.invoke(query)

for i, doc in enumerate(compressed_results):
    print(f"\n --- Result {i+1}-- ")
    print(doc.page_content)

```

Even though every source document is a full paragraph mixing multiple topics, the returned results are short, single-sentence answers — only the photosynthesis-related line from each document. Unrelated content about the Grand Canyon, castles, basketball, or cinema is filtered out entirely by the compressor before the results are returned.

COMMON MISTAKE

Assuming this retriever is unnecessary because "my documents would never mix unrelated topics." In practice, applying a text splitter to large source documents can easily produce chunks that straddle a paragraph boundary, mixing two topics into one chunk without the developer intending it.

Quick Revision — Contextual Compression Retriever

- Improves retrieval quality by compressing documents after retrieval, keeping only query-relevant content.
- Composed of two parts: a base retriever (fetches full documents) and a compressor (an LLM that trims them).
- Useful when documents are long, mix multiple topics, or when reducing LLM context length matters.
- Built with `ContextualCompressionRetriever(base_retriever=..., base_compressor=...)`.
- `LLMChainExtractor` is a common compressor implementation used to extract only relevant sentences.

8. Other Retrievers & Why So Many Exist

Beyond the five retrievers covered in depth in this chapter, LangChain provides many more, including:

- **Parent Document Retriever**
- **Time-Weighted Vector Retriever**
- **Self-Query Retriever**
- **Ensemble Retriever**
- **Multi Retriever**

Covering every retriever exhaustively is not practical in a single lesson. The official LangChain documentation lists all available retrievers along with sample code and descriptions, and is the right place to look up a specific retriever when a project calls for it.

WHY DO SO MANY RETRIEVERS EXIST?

Each retriever exists to solve a specific, narrow problem encountered when building RAG-based systems. From a Generative AI engineering perspective, the typical workflow is:

1. Build a simple RAG system using a basic (similarity search) retriever.
2. Observe that retrieval quality is not good enough — retrieved results may be redundant, off-topic, or noisy.
3. Swap the existing retriever for a more advanced one (MMR, Multi Query, Contextual Compression, or another) in the hope of improving retrieval quality.

Because retrievers are `Runnables` with a consistent interface, this swap can typically be made without restructuring the rest of the application.

INTERVIEW TIP

If asked "why does LangChain have so many retrievers?", frame the answer around RAG system improvement: different retrievers solve different retrieval-quality problems (redundancy, ambiguity, mixed-topic noise), and advanced RAG techniques are essentially built by choosing and combining the right retrievers for the observed weaknesses in a system.

Quick Revision — Other Retriever

- Many more retrievers exist beyond the five covered here (Parent Document, Time-Weighted, Self-Query, Ensemble, Multi Retriever, etc.).
- The official LangChain documentation is the reference for retrievers not covered in this chapter.
- Retriever exist in such variety because each solves a specific retrieval-quality problem.
- Improving a RAG system commonly means swapping in a more advanced retriever, not rebuilding the whole pipeline.
- Because retrievers are `Runnables`, swapping retrievers requires minimal changes to surrounding code.

K. Key Takeaways

- **Retriever**: a `Runnable` component that fetches relevant documents from a data source given a user query, behaving like a function from query to Document objects.
- **Categorization**: retrievers differ either by data source (Wikipedia, vector store, Arxiv) or by search strategy (MMR, Multi Query, Contextual Compression).
- **Wikipedia Retriever**: fetches Wikipedia articles using keyword-based matching, not semantic search.
- **Vector Store Retriever**: the most common retriever; performs semantic similarity search over embeddings, and its real value is being swappable for advanced strategies since it is a `Runnable`.
- **MMR**: reduces redundancy by picking results that are relevant yet dissimilar from already-selected results, tuned via `lambda_mult`.
- **Multi Query Retriever**: resolves query ambiguity by using an LLM to generate multiple reformulated queries, then merging and de-duplicating results.

- **Contextual Compression Retriever:** uses a base retriever plus an LLM compressor to trim retrieved documents down to only the query-relevant content.
- **Bigger picture:** advanced RAG systems are largely built by selecting and combining the right retrievers to fix specific retrieval-quality problems.

R. Revision Sheet

Retriever

A Runnable that takes a user query and returns relevant Document objects, performing internal search rather than blind loading.

Categorization Axes

By data source (what it searches) or by search strategy (how it searches) — two independent ways to classify any retriever.

Wikipedia Retriever

Queries the Wikipedia API using keyword matching to fetch relevant articles as Documents.

Vector Store Retriever

Performs embedding-based semantic similarity search over a vector store; created via `vector_store.as_retriever()`.

MMR

Maximum Marginal Relevance; balances relevance and diversity via `lambda_mult` (0 = max diversity, 1 = pure similarity).

Multi Query Retriever

Uses an LLM to expand an ambiguous query into multiple queries, then merges and de-duplicates results from each.

Contextual Compression Retriever

Base retriever + LLM compressor; keeps only query-relevant content from each retrieved document.

I. Interview Questions

- Q1.** What is a retriever in LangChain, and how does it differ from a document loader?
- Q2.** Why are all retrievers in LangChain implemented as `Runnable`s, and why does that matter?
- Q3.** What are the two broad ways to categorize retrievers, and give one example of each?
- Q4.** What search mechanism does the Wikipedia Retriever use internally?
- Q5.** If a vector store already supports `similarity_search()`, why would you still wrap it in a retriever?
- Q6.** Explain how Maximum Marginal Relevance reduces redundancy in retrieved results.
- Q7.** What does the `lambda_mult` parameter control in MMR, and what happens at its two extremes (0 and 1)?
- Q8.** How does the Multi Query Retriever handle ambiguous user queries?
- Q9.** Describe the two components that make up a Contextual Compression Retriever and what each does.
- Q10.** Why might a real-world document end up mixing multiple unrelated topics, and how does Contextual Compression address this?

F. Further Reading

- LangChain official documentation — Retrievers section (covers all built-in retriever types with sample code).
- LangChain documentation — `MultiQueryRetriever` reference.
- LangChain documentation — `ContextualCompressionRetriever` and `LLMChainExtractor` reference.
- LangChain documentation — Parent Document Retriever, Self-Query Retriever, and Ensemble Retriever (advanced retrievers not covered in depth in this chapter).